



Compute With Memory: Adapter Residency as a Price Term

Zach Kelling

Hanzo AI

2026

Abstract

Proof of work rests on an assumption that is easy to miss because it is never stated: hashpower is memoryless. Any miner is interchangeable with any other miner at the same rate, and no miner is cheaper for one particular block than for another. That assumption is what permits a single scalar difficulty.

Personalized agents break it. When an agent carries its own low-rank weight delta, a provider that has served that agent before is permanently cheaper for that agent than an otherwise identical provider that has not. Compute acquires memory, the supply side becomes non-fungible per buyer, and cost stops being a function of the endpoint and becomes a function of the path taken through resource space. That is precisely the condition under which a scalar price fails and a Hamiltonian formulation earns its keep.

We quantify the effect. A rank-16 adapter on a 70B model is 168 MB against a 140 GB base — a residency asymmetry of roughly three orders of magnitude. We derive the resulting cost functional, show that the naïve one-process-per-adapter deployment costs 175 accelerators where a batched deployment costs two, and state the single serving primitive that separates those two regimes. We report the exact state of that primitive in the Hanzo inference engine, including what is absent.

Contents

1	The memorylessness assumption	3
2	Where the memory comes from	3
2.1	Low-rank deltas	3
2.2	Parameter count and size	3
2.3	The asymmetry	4
3	The cost functional	4
4	Affinity, and what it implies for the market	4
5	Capacity: how many agents fit	5
6	What exists, and the one thing that does not	5
6.1	Training: complete	5
6.2	Serving: co-residency yes, selection no	6
6.3	The missing primitive	6
7	The same trick one layer down	6
8	Consequence for a Hamiltonian market	7

1 The memorylessness assumption

Bitcoin’s difficulty adjustment is a scalar because the resource it prices is one-dimensional and history-free. Write h_i for miner i ’s hash rate. The expected time for miner i to find a block at difficulty D is D/h_i , independent of which block, which transactions it contains, and everything miner i has ever done before. Two consequences follow, and both are load-bearing:

- (a) *Fungibility*. Miners at equal h are perfect substitutes. There is no buyer-specific cost.
- (b) *Statelessness*. The cost of the next unit of work does not depend on the sequence of previous units. There is no path dependence.

Together, (a) and (b) collapse the market to a single price per unit hash and a single difficulty. Neither survives contact with personalized inference.

2 Where the memory comes from

2.1 Low-rank deltas

A low-rank adaptation replaces a frozen projection $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ with

$$W'x = Wx + s \cdot BAx, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, B \in \mathbb{R}^{d_{\text{out}} \times r}, s = \frac{\alpha}{r}, \quad (1)$$

where $r \ll \min(d_{\text{in}}, d_{\text{out}})$. The pair (A, B) is the agent’s private state. Everything else is shared.

This is the shape implemented in `hanzo-train/src/lora.rs`, where the base is a frozen `Linear` and the delta is a separate tracked `LoraDelta { name, a, b, scale }` carrying A as (r, d_{in}) and B as (d_{out}, r) .

2.2 Parameter count and size

Adapting T projections in each of L layers of a model with hidden dimension d costs

$$|\theta_{\text{adapter}}| = 2rLTd \quad (2)$$

parameters when the adapted projections are square in d . At `bf16` the byte cost is twice that. Table 1 gives the values that matter.

Model class	Targets	r	Parameters	Size (bf16)
7B ($d=4096, L=32$)	q,k,v,o	8	8,388,608	16.8 MB
7B	q,k,v,o	16	16,777,216	33.6 MB
7B	q,k,v,o + mlp	16	29,360,128	58.7 MB
7B	q,k,v,o	64	67,108,864	134.2 MB
70B ($d=8192, L=80$)	q,k,v,o	8	41,943,040	83.9 MB
70B	q,k,v,o	16	83,886,080	167.8 MB
70B	q,k,v,o	64	335,544,320	671.1 MB

Table 1: Adapter parameter counts and sizes.

2.3 The asymmetry

Set against those figures, the base model is 140 GB at **bf16** for a 70B, or roughly 40 GB quantized to four bits. The ratio that governs everything downstream is

$$\frac{|\text{base}|}{|\text{adapter}|} \approx 10^3. \quad (3)$$

An adapter is nearly free to move; the base it attaches to is, for practical purposes, immovable within any interactive latency budget. At 10 Gbit s^{-1} a 140 GB transfer takes 112 s. Against a 3 s service objective that is not expensive — it is infeasible. Feasibility and price are therefore entangled through residency, which is the technical heart of this paper.

3 The cost functional

Definition 1 (Residency state). *For a provider R at time t , let $\beta(R, t) \in \{0, 1\}$ indicate whether the base model required by job J is resident in accelerator memory, and $\alpha(R, J, t) \in \{0, 1\}$ whether J 's adapter is resident.*

The allocation objective in the heterogeneous market is then

$$\mathcal{C}(J, R) = \underbrace{p(R) \tau(J, R)}_{\text{compute}} + \lambda_L \ell + \lambda_F \phi + \lambda_V v + \underbrace{\mu_1(1 - \beta) + \mu_2(1 - \alpha)}_{\text{cold-start}}, \quad (4)$$

with ℓ latency, ϕ failure risk, v verification cost, and

$$\frac{\mu_1}{\mu_2} \approx \frac{|\text{base}|}{|\text{adapter}|} \approx 10^3. \quad (5)$$

The last two terms are what a stateless spot market omits, and they are not a correction — at interactive deadlines μ_1 is frequently large enough to make the constraint binding rather than the price.

Proposition 1 (Cheapest capable is wrong). *Let R_1 be an accelerator priced at p_1 with $\beta = 1$, and R_2 priced at $p_2 \ll p_1$ with $\beta = 0$. If the deadline Δ satisfies $\Delta < |\text{base}|/bw(R_2)$, then R_2 is infeasible regardless of p_2 , and the correct allocation is R_1 at arbitrarily large price ratio.*

The worked instance: an accelerator at \$4/hour holding the model beats one at \$0.50/hour that must first fetch 140 GB. Under a 3 s objective the cheaper machine is not in the feasible set at all.

4 Affinity, and what it implies for the market

Because $\mu_1 \gg \mu_2$, agents develop *affinity* for the nodes that already hold their base model, and their adapter follows cheaply. This produces structure a spot market cannot express:

Stickiness is efficient, not rent-seeking. Routing an agent back to the same provider is the cost-minimising choice, not a lock-in artifact. A market design that forces uniform random allocation across capable providers destroys real surplus.

Eviction is a pricing decision. An accelerator holding k resident adapters faces a knapsack problem whenever a new agent arrives. The shadow price of a residency slot is the expected discounted future value of the agent occupying it, which is exactly the object a continuous market can quote and a discrete order book cannot.

Residency is self-proving. A provider cannot profitably lie about holding an agent’s weights, because the lie is revealed by the deadline it then misses. This is unusual and useful: the market needs no attestation mechanism for this particular claim.

State rent is a new instrument. An agent may pay a small continuous fee to keep its adapter resident somewhere, converting a stochastic cold-start cost into a predictable stream. This is an option on locality. It has no analogue in a stateless compute market and it is a revenue line that does not exist today.

5 Capacity: how many agents fit

The reason this matters economically is that adapters are small enough that personalization is not, in fact, expensive — once the delta stops dragging a base model behind it.

Base	Free memory	Adapter	Resident agents
70B bf16 on 180 GB	40 GB	17.2 MB	2,326
70B bf16 on 180 GB	40 GB	137.4 MB	291
70B 4-bit on 80 GB	40 GB	17.2 MB	2,326
7B bf16 on 80 GB	65 GB	17.2 MB	3,779

Table 2: Resident agents beside one shared base, ignoring KV cache.

Against that, the cost of *not* sharing the base:

100 personalized agents, 70B bf16	Memory	Accelerators
One process per adapter	14,000 GB	175
Batched multi-adapter	141.7 GB	2

Table 3: The duplication cost, at 80 GB per accelerator.

An $\approx 87\times$ reduction in the capital required to serve a personalized population is not an optimization. It sets the cost floor for the entire category, and whoever ships it first defines that floor for everyone else.

6 What exists, and the one thing that does not

We state the current position precisely, because the gap is narrow and specific and it is worth no one’s time to mistake it for a research problem.

6.1 Training: complete

hanzo-train is 2,215 lines implementing trainable LoRA over frozen bases, AdamW, distillation, a data loader, and PEFT-format serialization whose tensor names match the inference-side merge. It is exposed over the serving API:

```
POST /v1/training/clients base model + LoRA injection
GET /v1/training/clients/{id} list / inspect (+ loss history)
POST /v1/training/clients/{id}/forward_backward accumulate gradients
POST /v1/training/clients/{id}/optim_step apply AdamW
POST /v1/training/clients/{id}/sample decode from current base+delta
POST /v1/training/clients/{id}/save_weights write the adapter for inference
DELETE /v1/training/clients/{id} drop client, free memory
```

An agent can be given its own weights today, end to end.

6.2 Serving: co-residency yes, selection no

The inference side carries `hanzo-engine/src/lora/loralinear.rs` and `qloralinear.rs`, an X-LoRA implementation under `hanzo-engine/src/qlora_models/`, and AnyMoE. X-LoRA is significant here for a reason unrelated to its purpose: it loads many adapters into one process simultaneously. *Co-residency is solved*. What X-LoRA does with those adapters — mix all of them for every token through a learned classifier — is not what a market needs, but the memory management, shape handling and loading are done.

What is absent is selection:

- `preload_adapters` is a load-time `HashMap`, threaded through the loaders as `Option<HashMap<String, (ShardedVarBuilder, LoraConfig)>>`.
- `with_lora(adapters: Vec<String>)` is a pipeline-construction argument.
- `hanzo_quant::lora::merge_lora_weights` folds the delta *into* the base weight at load, which destroys per-adapter separability by design.
- The OpenAI-compatible request in `hanzo-server-core/src/openai.rs` carries no adapter field.

One process therefore serves one adapter configuration, which is exactly the regime that costs 175 accelerators in Table 3.

6.3 The missing primitive

Batched multi-adapter serving requires that a single forward pass over a batch of sequences apply a *different* delta per sequence:

$$y_i = Wx_i + s_{a(i)} B_{a(i)} A_{a(i)} x_i, \quad a(i) \in \{1, \dots, k\}, \quad (6)$$

where $a(i)$ is the adapter index of sequence i in the batch. Implemented as a segmented gather-matmul, the added cost over the shared base is $O(r(d_{\text{in}} + d_{\text{out}}))$ per token — negligible against the $O(d_{\text{in}} d_{\text{out}})$ base projection at $r \leq 64$.

The unmerged forward already exists in the tree, on the training side: `LoraLinear` in `hanzo-train/src/lora.rs` keeps `base` frozen and `delta: Option<LoraDelta>` separate, computing $s(xA^\top)B^\top$ as its own path with tensor layouts documented as matching the inference merge. The work is to make that path batch-aware over $a(i)$, add an adapter field to the request, and place a cache with eviction in front of it. Every piece has a reference implementation in the same repository.

7 The same trick one layer down

The pattern in this paper — a large frozen shared object plus a tiny per-tenant delta — is not confined to model weights. Our pricing layer uses it too, and the symmetry is worth naming because it suggests the shape is general rather than an artifact of how transformers happen to be built.

There, tenants differ in how they trade price against latency and quality. The straightforward implementation is a full-precision preference table per tenant, which does not survive scale. Instead the tenant’s deviation is carried as a one-bit sign delta over a shared base quote table — one bit per regime and resource cell, with per-layer scale factors and higher-precision bias terms retained. The compression against a single-precision table is thirty-two-fold, which is the difference between holding thousands of tenant tables in memory and holding millions.

Two layers, same structure: the shared object is large and effectively immovable, the per-tenant object is tiny, and the economics of the whole system fall out of that ratio. Where the analogy stops is instructive. At the pricing layer the shared table is small enough that duplicating it per tenant is merely wasteful; at the model layer the shared object is a hundred and forty gigabytes and duplicating it is the difference between two accelerators and a hundred and seventy-five. The asymmetry is what makes selection — rather than duplication — the load-bearing primitive at the model layer specifically.

8 Consequence for a Hamiltonian market

With stateless homogeneous inference, a spot market for accelerator-hours is close to sufficient; the resource manifold is nearly a line and a price vector nearly spans it. Personalization is what makes the manifold genuinely coupled: the cost of serving a request depends on the trajectory the system has already taken, because that trajectory determines what is resident where.

This is the concrete content of treating compute as a coupled resource system rather than a family of independent order books. The coupling is not between device types — those really are close to substitutes at equal capability. The coupling is between *a buyer's history* and *a provider's current state*, and it is strong, asymmetric, and continuously varying. A scalar difficulty cannot express it. A Hamiltonian in (q, p) over occupancy and price can.

Availability

The inference engine, training crate, router and model catalog discussed here are open source under permissive licence. The adapter arithmetic in Tables 1–3 is reproducible from the formulas given; no proprietary measurement is involved.